Academic Year 2025 – 2026
Research Unit: Delta Informatica / Saidea / Diventa S.r.l.
Team(s): Research & Development
Supervisor(s): A. Zorer (company) & D. Miorandi (academic)

UNIVERSITÀ DI TRENTO

DATA SCIENCE — MSC

Final Internship Report

Ettore Miglioranza

April 2026

Abstract

This report documents the internship carried out at Delta Informatica / Saidea / Diventa S.r.l., an ICT services company based in Trento, Italy, from February 25 to April 10, 2026 (250 hours). The internship was hosted within the Research & Development department, under the supervision of Alessandro Zorer (company tutor) and Daniele Miorandi (academic tutor).

The project developed during the internship — *Antares AI* — is a secure, privacy-preserving system for the semantic routing of natural language user requests through a Large Language Model (LLM). The core challenge was integrating a public LLM into a business workflow without exposing any personally identifiable information (PII) to the model. The solution implements a reversible token-based anonymization pipeline: PII is detected and replaced with pseudonymous tokens before crossing the LLM boundary, the model performs intent classification on the anonymized text only, and original data is restored exclusively server-side at the point of tool execution.

The system is composed of four cooperating services: a .NET MCP server exposing business tools to the Claude LLM, a custom anonymization API backed by a GLiNER named-entity recognition model fine-tuned for Italian, a Redis-based session context store, and a .NET MCP client orchestrating the full request cycle. Three real-world workforce management use cases were implemented and validated. The internship followed a fluid, iterative process culminating in a final demo presented to the company team and management.

Keywords: LLM integration, MCP protocol, privacy-by-design, PII anonymization, .NET, AI agents, semantic routing, C#

Contents

1 The Host Organization

1.1 Company Profile

Delta Informatica / Saidea / Diventa S.r.l. is an Italian ICT services company headquartered in Trento. The company specializes in the outsourcing of business information systems and offers a broad portfolio of services including IT infrastructure management, help desk, desktop management, custom software development, and strategic IT consulting. Where a client already has an internal IT department, the company provides targeted support for specific functions, integrating seamlessly into existing structures.

What distinguishes Delta Informatica / Saidea / Diventa S.r.l. from conventional ICT vendors is its consulting philosophy: rather than delivering products and services in isolation, the company positions itself as the client's *external IT department*. This means investing deeply in understanding the client's business processes, identifying operational gaps and strengths, and defining customized strategic IT solutions aligned with the client's actual needs. This approach is encapsulated in the company's own description: "we don't just provide software and hardware — we learn your company, identify its weaknesses and strengths, and indicate the strategic moves to reach success."

The internship was conducted within the Research & Development area, where the company invests in exploring novel software architectures and emerging technologies to extend its service offering.

1.2 Working Environment and Team

The team is deliberately small and specialized, consisting of four operative developers and a Chief Technology Officer (CTO). This lean structure favors technical depth, rapid iteration, and direct communication across all levels of seniority.

The working atmosphere was dynamic and open: technical confrontations and architectural discussions were the norm rather than the exception, and every member of the team proved consistently available and supportive. While the organizational structure followed a corporate model — with formal review cycles and managerial oversight — the day-to-day culture was approachable and genuinely collaborative. The relationship with the company tutor, Alessandro Zorer, was positive and productive throughout the internship, characterized by continuous oversight, honest feedback, and active participation in key project milestones.

2 Activities Carried Out

2.1 Project Overview: Antares AI

The internship project, *Antares AI*, addresses a concrete challenge that arises when integrating public Large Language Models into enterprise software workflows: how to leverage the semantic understanding capabilities of a state-of-the-art LLM for intent classification and request routing without transmitting any sensitive business or personal data to an external service.

The context is a workforce management platform (Antares/Miorelli) used by companies to manage personnel assignments, construction site operations, and absence tracking. Users interact with the system through natural language requests in any language, which must be interpreted, routed to the correct business operation, and executed — all while complying with strict data protection requirements set by management.

The solution developed is a four-component distributed system built around the *Model Context Protocol* (MCP), a standardized interface for exposing business tools to LLM-based orchestrators.

2.2 System Architecture

The system comprises four cooperating runtime services:

— **Antares MCP Server** (C#/.NET 10, port 5001): exposes a set of typed business tools to the Claude LLM via the MCP protocol over HTTP/SSE transport. The primary tool, `ask_antares`, acts as a semantic router: it receives an anonymized user prompt, invokes Claude with a multi-language system prompt, and delegates execution to the appropriate business tool based on the model's intent classification.

— **Anonymization Service API** (C#/.NET 10, port 5292): implements the core privacy mechanism. On the input path, it calls the GLiNER service to detect named entities, replaces them with reversible tokens (e.g., <PERSON_0001>, <LOCATION_0001>), and stores the token-to-original mapping in Redis under a unique context identifier. On the output path, it restores original values from the stored mapping.

— **GLiNER Service** (Python/FastAPI, port 5100): serves a multilingual named-entity recognition model (`urchade/gliner_multi-v2.1`) trained to detect persons, organizations, locations, phone numbers, emails, and dates in natural language text. Entities are filtered by confidence score, minimum span length, and a domain-specific stopword list to minimize false positives.

— **MCP Client** (C#/.NET 10): orchestrates the complete request cycle. It constructs the natural language prompt, calls the Anonymization API to obtain anonymized text and a context identifier, invokes `ask_antares` on the MCP Server with the anonymized payload, and returns the final result.

The architectural invariant that governs the entire system is: *no PII crosses the MCP boundary in cleartext*. Token maps are held exclusively in Redis, deanonymization occurs only server-side at the point of tool execution, and the client receives only a controlled, privacy-safe response.

2.3 Work Phases and Methodology

The internship followed a fluid, iterative process shaped by the progressive availability of information and by the evolving requirements characteristic of a real R&D project.

Phase 1 — Requirements and Architecture Design. The first weeks were dedicated to a thorough analysis of the problem domain: defining the project goals, eliciting functional and non-functional requirements (with particular emphasis on privacy and security), selecting the technology stack, and designing

the system architecture. This phase culminated in a formal presentation to the company team and management on March 12, 2026, in which the full architectural proposal was exposed, discussed, and ultimately approved. The session generated rich technical feedback and set the direction for the subsequent implementation.

Phase 2 — Infrastructure Build-up. With the architecture approved, the focus shifted to establishing the development environment and building the foundational infrastructure: service scaffolding, inter-service communication, dependency injection configuration, MCP transport setup, and Redis context storage.

Phase 3 — Use Case Implementation and Iterative Refinement. Once the real-world use cases were received from the company, the infrastructure was shaped around concrete business requirements. This phase proceeded in iterative cycles of development, functional benchmarking, architectural refinement, and requirement adaptation until a fully working demo was produced. The three implemented use cases are:

- **UC-A**: given a construction site name, find and rank the nearest available workers within a configurable radius, using Haversine distance computation;
- **UC-B**: given a person's name, identify their colleagues (workers assigned to the same sites) and rank nearby non-colleagues by proximity;
- **UC-C**: analyze absence coverage for a holiday request, checking colleagues, nearby substitute workers, and the authorization chain (responsible manager, authorizer).

The final demo was presented to the company team and management in April 2026.

2.4 Technical Challenges and Design Decisions

Several significant challenges arose during implementation, each requiring a deliberate design response.

Technology Stack Adoption. The project required proficiency in C# and the .NET 10 ecosystem — a mature, enterprise-grade development platform with a substantial learning curve for someone with primarily Python and academic programming experience. Beyond the language itself, the challenge involved understanding the existing codebase, learning its architectural conventions, and contributing to it in a way that was idiomatic and maintainable.

Security and Privacy Requirements. Fulfilling the data protection requirements set by management was one of the most demanding aspects of the project. Privacy-by-design had to be applied not merely as an add-on feature but as a fundamental architectural constraint, with every component designed to minimize data exposure. The cybersecurity implications — token isolation, session scoping, server-side-only deanonymization, and resilient error handling — required constant attention throughout development.

Anonymization Technology Pivot: Presidio to GLiNER. The initial choice of Microsoft Presidio as the PII detection engine proved insufficient when confronted with the actual Multi-language use cases. Presidio's entity recognition was not flexible enough for the domain-specific vocabulary and the linguistic characteristics of the real inputs. After receiving the concrete use cases, the anonymization module was redesigned around a custom pipeline built on the GLiNER transformer model, which offered significantly better recall and flexibility for Italian NER tasks.

Real Backend Integration Deferred. The full integration with the Miorelli production database and

its REST API was not completed during the internship. The production database contains large volumes of highly sensitive personnel data; building the anonymization layer and the API integration from scratch required the full team's attention and careful, time-consuming design to avoid any risk of data leakage. The team collectively decided to prioritize security and correctness over delivery speed. The system therefore operates on a mock data provider, with a clean abstraction layer (`IMiorelliService` / `MiorelliServiceStub`) designed for a single-line substitution when the real client becomes available.

Voice Input Deferred. Voice input was included in the original project scope as an optional extension. Given the constraints of the available timeframe, and with the working text-based demo fully validated, the voice channel was pragmatically deprioritized as a future add-on. The decision reflected a considered trade-off between scope and delivery quality.

3 Objectives and Degree of Achievement

The internship project defined five formative objectives in the official project agreement. The following assessment maps each objective to the actual outcomes.

Objective 1: Understand LLM principles and their responsible use within secure software architectures. *Fully achieved.* Claude Sonnet 4.6 was integrated as the semantic routing engine of the system. The integration was designed with explicit constraints: the model receives only anonymized, context-stripped prompts; its output is limited to tool selection and parameter extraction; and the system prompt enforces strict privacy instructions in Italian. The internship provided deep, hands-on understanding of LLM capabilities, limitations, and responsible integration patterns.

Objective 2: Develop semantic analysis and classification of user requests in multilingual and multi-channel scenarios. *Substantially achieved.* Semantic classification of Multi-language natural language requests was fully implemented and validated across three distinct use cases. Voice input, representing the "multi-channel" dimension of this objective, was not implemented within the internship timeframe and was explicitly deferred as a future extension.

Objective 3: Design software flows minimizing data transmission, in compliance with privacy-by-design and data protection criteria. *Fully achieved.* Data minimization is the defining architectural principle of the system. No PII is transmitted to the external LLM at any stage: the anonymization pipeline ensures that only tokenized, identity-stripped text crosses the MCP boundary. Session context is stored ephemerally in Redis and is not persisted beyond the lifecycle of a single request.

Objective 4: Integrate heterogeneous tools (local scripts, MCP services, AI models) within a coherent, secure, and scalable workflow. *Fully achieved.* The delivered system integrates four services across two programming ecosystems (C#/.NET and Python/FastAPI), a Redis cache, a transformer-based NER model, the Claude API, and the MCP protocol — all operating as a coherent, secure, and demonstrably functional workflow.

Objective 5: Develop autonomy in technical documentation, design evaluation, and functional validation. *Fully achieved.* Two formal presentations were delivered to the company (initial proposal and final demo). Technical documentation of the architecture and workflow was produced in Markdown. The system was validated through iterative functional testing against real and simulated business scenarios.

4 Final Considerations

4.1 Educational Value

The internship delivered a breadth of technical learning that would have been difficult to replicate in an academic setting. The most significant acquisitions were:

C# and the .NET Ecosystem. Going from limited prior exposure to functional proficiency in C# and .NET 10 within the span of a few weeks was one of the most demanding and rewarding aspects of the experience. Beyond the language itself, this meant learning the patterns of a mature, enterprise-grade development ecosystem: dependency injection, strongly-typed HTTP clients, NuGet package management, multi-project solutions, and the conventions of professional C# codebases.

Software Engineering in Practice. The internship made concrete a distinction that university coursework tends to obscure: the difference between solving a well-defined algorithmic problem and building a system under real-world constraints. Complexity in the workplace does not come primarily from algorithms; it comes from integration, from requirements that change mid-implementation, from existing codebases with their own conventions, and from operational concerns (security, reliability, maintainability) that rarely feature in academic assignments.

Cybersecurity Applied. The privacy-by-design requirements moved cybersecurity from theoretical knowledge to a continuous engineering constraint. Concepts learned in the university's cybersecurity course — PII protection, data minimization, access control — were applied directly and non-trivially in the design of the anonymization pipeline and the MCP communication boundaries.

AI Agents and the MCP Protocol. The MCP protocol and the broader paradigm of LLM-based tool orchestration (AI agents) required fully independent study, as no university coverage of these topics existed. This demonstrated the importance of self-directed learning and the ability to rapidly master emerging technologies from primary documentation and experimental implementation.

4.2 University Preparation

Several courses from the Data Science curriculum proved directly valuable in ways that were often unexpected in their practical applicability.

The *Big Data Technologies* course provided the most transferable foundation: system design principles, database modeling, API consumption patterns, scheduling, and the general mental model of distributed, scalable systems were all directly applicable. The *Machine Learning* course provided the conceptual grounding necessary to understand, deploy, and critically evaluate a transformer-based NER model. The *Cybersecurity* course proved essential for the privacy and security dimensions of the project. The *High-Performance Computing* course contributed useful background in networking and distributed execution. The *Scientific Programming* course and several programming-focused courses established the general coding competence that made rapid adaptation to a new language and framework possible.

The most significant gaps in preparation were in application development (web applications, client-server architecture, REST API design) and in the depth of networking knowledge required for production sys-

tems. Both had to be acquired independently on the job. The MCP protocol and AI agent architectures represented an entirely uncovered area, reflecting how rapidly the AI engineering landscape is evolving relative to the pace of curriculum updates.

4.3 Relational Experience and Soft Skills

Work was organized in a balanced split between autonomous development and close collaboration with senior developers, who contributed substantially to technical learning through direct mentorship and code review. The company tutor maintained consistent oversight throughout the project, with regular check-ins that kept the work aligned with business requirements and quality standards.

The **initial presentation** on March 12 was a defining relational and professional milestone. Presenting an architectural proposal — including requirements analysis, design decisions, and implementation roadmap — to a room of experienced developers and company management was a genuinely challenging experience. The initial apprehension was quickly replaced by engagement, as the session evolved into a substantive technical discussion with thoughtful feedback on the proposed architecture. The experience of defending design choices under real professional scrutiny, incorporating feedback in real time, and obtaining a formal project approval was unlike anything encountered in academic settings.

The **final demo presentation** in April carried a different weight: it was not a proposal to be evaluated but a working system to be demonstrated. Presenting a validated, functional artifact to the same audience that had reviewed the initial proposal six weeks earlier made concrete the full arc of the internship and the distance traveled from design to delivery.

More broadly, the internship underscored that software engineering is not a purely technical discipline. The ability to communicate clearly with peers, to listen carefully and translate informal business requirements into precise technical specifications, and to explain architectural decisions to non-technical stakeholders proved to be as important as any programming skill. These are competencies that can only be developed through genuine professional exposure.

5 Satisfaction

The internship delivered precisely what was hoped for when entering it. The expectation was to confront a real-world project with difficult, evolving requirements; to work alongside colleagues and senior engineers in building a genuine solution; to master new technologies and frameworks; to develop communication and presentation skills; and to close the gap between theoretical university knowledge and its practical application. All of these expectations were met.

The two presentation moments stand out as the experiences of greatest personal and professional significance. The initial presentation — where a self-designed architectural proposal was reviewed and approved by a professional team — and the final demo — where a working system was delivered and demonstrated to company management — represented growth milestones that went beyond the technical dimension of the project. They were moments in which the full scope of what had been learned, designed, and built became visible at once.

The overall internship experience was positive in every dimension: technical, relational, and professional. The environment at Delta Informatica / Saidea was genuinely formative, and the project was sufficiently challenging to stretch capabilities without becoming unmanageable. The pragmatic decisions made along the way — deferring voice input, prioritizing data security over full backend integration, replacing Presidio with GLiNER — were not frustrations but exercises in the kind of judgment that real engineering requires.

Acknowledgements

Acknowledgements. This internship was carried out in the Research & Development department of Delta Informatica / Saidea / Diventa S.r.l., Trento, under the supervision of Alessandro Zorer and Massimo Turri, whom I would like to thank for their constant guidance, the quality of their supervision and their genuine support throughout the entire experience. I am grateful to the whole team for the welcoming environment, the willingness to share expertise, and the collaborative spirit that made this internship genuinely formative. I also wish to thank Professor Daniele Miorandi for the academic supervision and the University of Trento for the organizational support that made this experience possible.